

Docker & Docker Compose

DevOps Internship — Week 4 Report

Submitted by: Kosgei Kipngetich

Program: Davine Technologies — DevOps Internship

Week: Week 4

Topic: Docker & Docker Compose

Report Date: 18/08/2026

GitHub Repository: <https://github.com/Engkipngetich/week4-docker.git>

Week 4 Report — Docker & Docker Compose

Davine Technologies DevOps Internship

1. What is Docker?

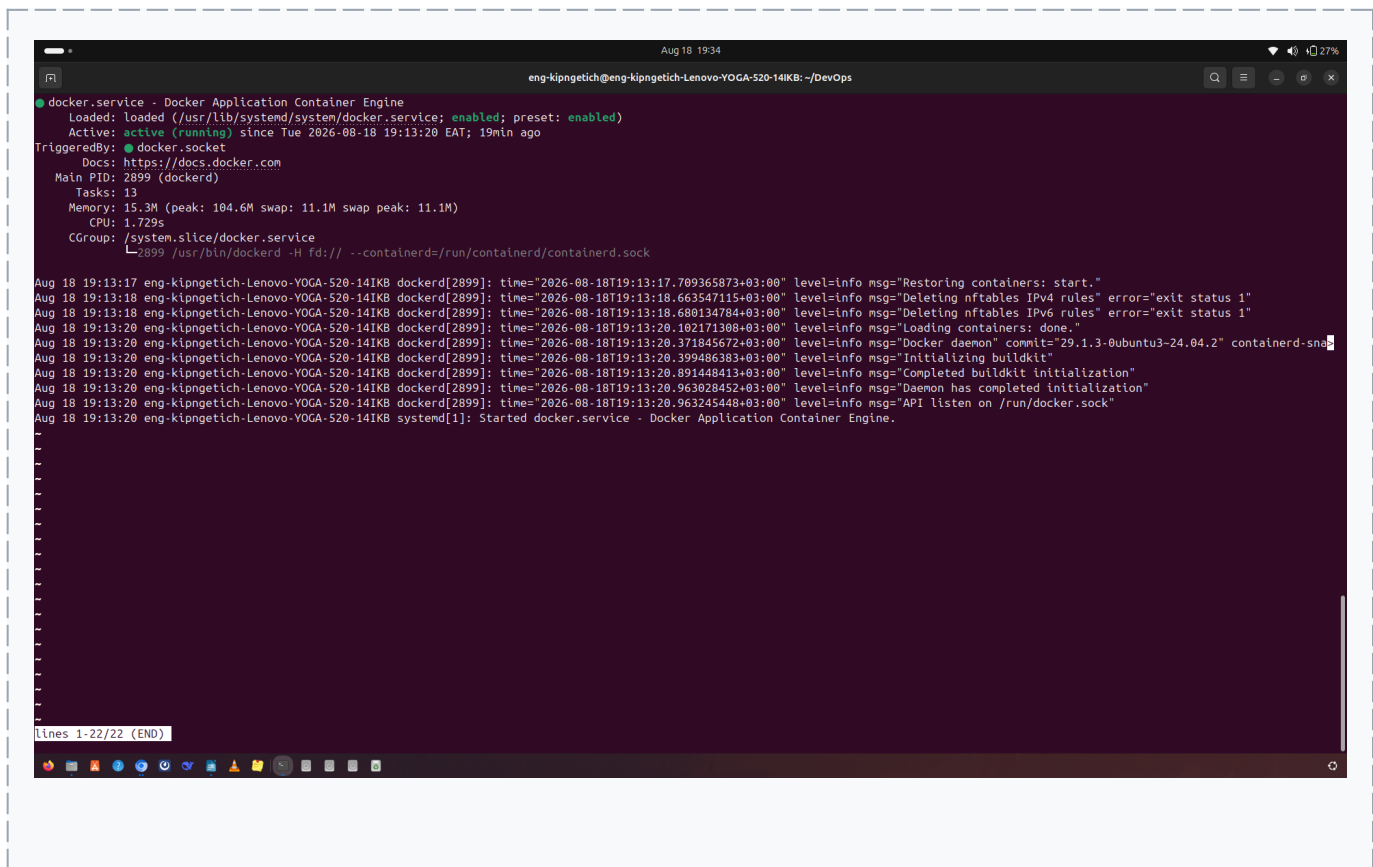
Docker is an open-source containerization platform that packages an application together with everything it needs to run — code, runtime, system libraries, and configuration — into a single, portable unit called a container. Because the container includes its own environment, it runs the same way regardless of where it is deployed: a developer's laptop, a test server, or a production cloud environment. This solves the classic "it works on my machine" problem and makes applications easier to build, ship, and run consistently.

2. Docker Architecture

Docker uses a client-server architecture made up of the following core components:

- **Docker Client:** The command-line tool (**docker**) used to issue commands such as build, run, and pull.
- **Docker Daemon (dockerd):** The background service that builds, runs, and manages containers, images, networks, and volumes.
- **Docker Images:** Read-only templates used to create containers.
- **Docker Containers:** Running instances of images.
- **Docker Registry (e.g. Docker Hub):** A remote repository for storing and distributing images.

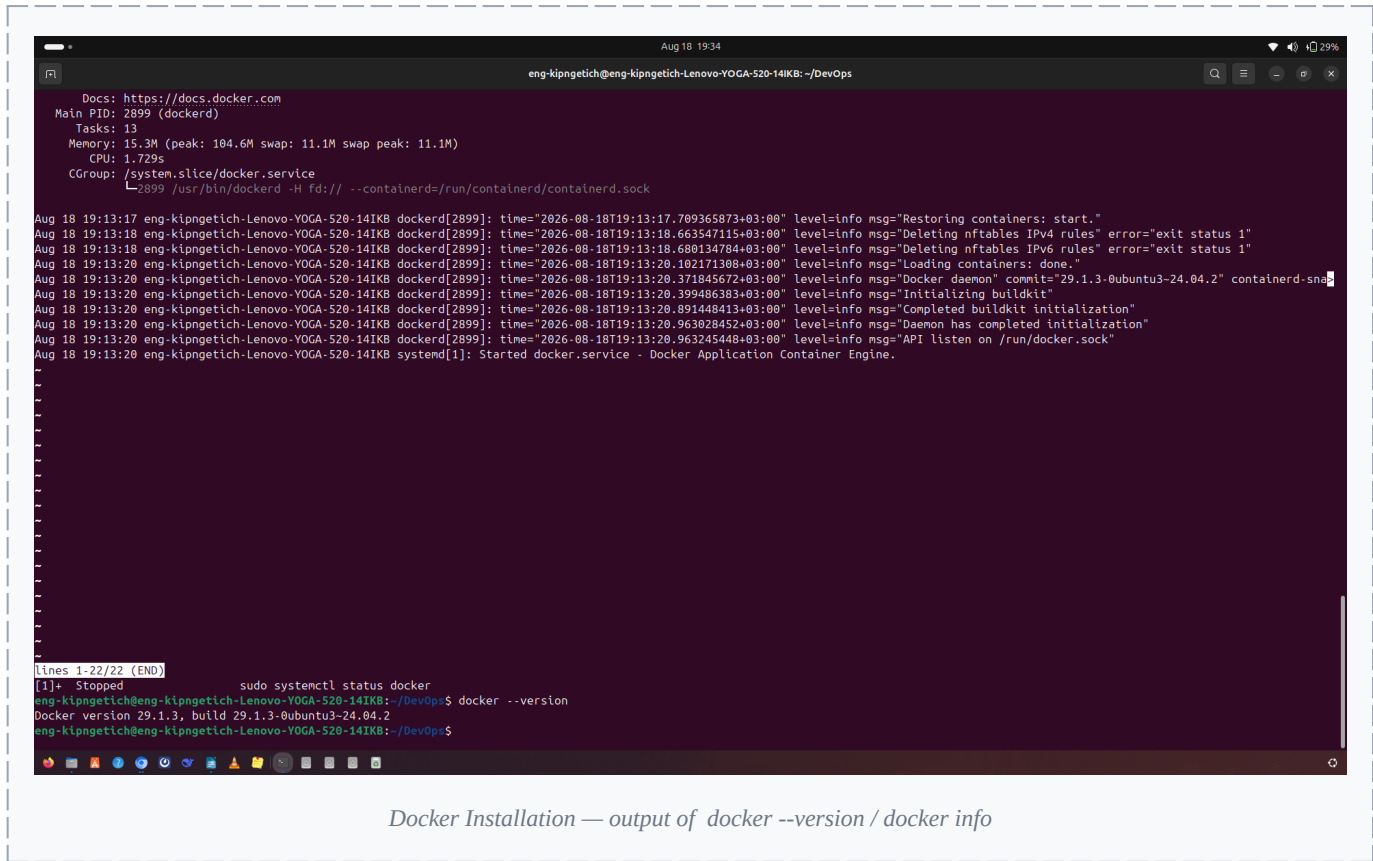
The client communicates with the daemon (locally or remotely) via a REST API, and the daemon carries out the requested actions on the host system.



```
Aug 18 19:34
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps

● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-18 19:13:20 EAT; 19min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2899 (dockerd)
      Tasks: 13
     Memory: 15.3M (peak: 104.6M swap: 11.1M swap peak: 11.1M)
        CPU: 1.729s
    CGroup: /system.slice/docker.service
            └─2899 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Aug 18 19:13:17 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:17.709365873+03:00" level=info msg="Restoring containers: start."
Aug 18 19:13:18 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:18.663547115+03:00" level=info msg="Deleting nftables IPv4 rules" error="exit status 1"
Aug 18 19:13:18 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:18.680134784+03:00" level=info msg="Deleting nftables IPv6 rules" error="exit status 1"
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.102171308+03:00" level=info msg="Loading containers: done."
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.371845672+03:00" level=info msg="Docker daemon" commit="29.1.3-0ubuntu3-24.04.2" containerd-sna
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.399486383+03:00" level=info msg="Initializing buildkit"
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.891448413+03:00" level=info msg="Completed buildkit initialization"
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.963028452+03:00" level=info msg="Daemon has completed initialization"
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB dockerd[2899]: time="2026-08-18T19:13:20.963245448+03:00" level=info msg="API listen on /run/docker.sock"
Aug 18 19:13:20 eng-kipnetich-Lenovo-YOGA-520-14IKB systemd[1]: Started docker.service - Docker Application Container Engine.
```



Docker Installation — output of `docker --version` / `docker info`

3. Images vs Containers

Docker Image	Docker Container
A static, read-only blueprint/template	A running (or stopped) instance of an image
Built once, reused many times	Created fresh from an image each time it is run
Stored in a registry (e.g. Docker Hub)	Exists on the host machine while running
Does not consume runtime resources	Consumes CPU, memory, and storage while active

In short: an image is the recipe, and a container is the dish made from that recipe.

```
Aug 18 19:55
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB:~/DevOps$ docker images

```

IMAGE	ID	DISK USAGE	CONTENTS	SIZE	EXTRA
las-project-react-front-app:latest	dc99cd366f7b	4.42GB		0B	
laimell/phpldapadmin:1.2.6.6	5303ea9877f1d	229MB		0B	
mongo-express:latest	870141b735e7	182MB		0B	
mongo:latest	f0286cc18189	942MB		0B	
nginx:alpine	b76de378d572	62.1MB		0B	
nginx:latest	5253dc86cc93	161MB		0B	
nvdiia/cuda:12.0.0-base-ubuntu22.04	ed4307630298	237MB		0B	
osixia/openldap:1.5.0	2da3cf664891	257MB		0B	
ubuntu:latest	86a1a31fdd84	108MB		0B	

```
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB:~/DevOps$
```

Docker Images — output of docker images

```
Aug 18 20:01
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB:~/DevOps$ docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b67685bebcc6	nginx	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	nginx-container-2
2302d0435626	nginx	"/docker-entrypoint..."	23 minutes ago	Up 23 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	my-nginx

```
eng-kipnetich@eng-kipnetich-Lenovo-YOGA-520-14IKB:~/DevOps$
```

Running Containers — output of docker ps

4. Dockerfile

A Dockerfile is a plain-text script containing step-by-step instructions for building a Docker image automatically. Common instructions include:

- **FROM** — specifies the base image
- **WORKDIR** — sets the working directory inside the container
- **COPY** — copies files from the host into the image
- **RUN** — executes a command during the build (e.g. installing dependencies)
- **EXPOSE** — documents the port the container listens on
- **CMD / ENTRYPOINT** — defines the command that runs when the container starts

Example Dockerfile for a simple Node.js application:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

5. Docker Volumes

Containers are ephemeral by default — data inside them is lost when the container is removed. Docker Volumes solve this by providing a persistent storage mechanism that exists independently of the container's lifecycle. Volumes are managed by Docker and can be shared across multiple containers, making them the recommended way to persist data such as database files.

```
# Create a named volume
docker volume create app_data

# Mount it into a container
docker run -d -v app_data:/var/lib/data my-image
```

6. Docker Networks

Docker Networks allow containers to communicate with each other and with the outside world in a controlled way. Docker provides several network drivers:

- **bridge** — the default network for standalone containers on a single host
- **host** — removes network isolation between the container and the host
- **none** — disables networking entirely
- **custom bridge networks** — allow containers to resolve each other by name, ideal for multi-container applications

```
# Create a custom network
docker network create app-network

# Attach a container to it
docker run -d --network app-network --name backend my-backend-image
```

7. Environment Variables & Port Mapping

Environment variables allow configuration (e.g. database credentials, API URLs) to be injected into a container at runtime without hardcoding them into the image. Port mapping exposes a container's internal port to the host machine so external traffic can reach the application.

```
docker run -d \
  -e DB_HOST=database \
  -e DB_PASSWORD=secret \
  -p 8080:3000 \
  my-app-image
```

8. Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications using a single YAML configuration file (docker-compose.yml). Instead of starting each container manually, Compose lets you define all services, networks, and volumes in one place and bring the entire stack up or down with a single command.

```
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=database
      - DB_PASSWORD=secret
    depends_on:
      - database
    networks:
      - app-network

  database:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - db_data:/var/lib/postgresql/data
    networks:
      - app-network

volumes:
  db_data:

networks:
  app-network:
```

```
# Bring the whole stack up
docker compose up -d --build

# Bring it down
docker compose down
```

9. Docker Hub

Docker Hub is Docker's default cloud registry for storing and sharing container images. It allows images to be pushed, pulled, and version-tagged, making it easy to distribute a built image to any machine with Docker installed.

```
# Tag and push an image
docker tag my-app-image myusername/my-app-image:v1
docker push myusername/my-app-image:v1
```

```
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app
-rw-rw-r-- 1 eng-kiipnetich eng-kiipnetich 124 Aug 18 20:28 Dockerfile
-rw-rw-r-- 1 eng-kiipnetich eng-kiipnetich 181 Aug 18 20:27 package.json
-rw-rw-r-- 1 eng-kiipnetich eng-kiipnetich 382 Aug 18 20:26 server.js
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app$ docker build -t my-node-app:v1 .
[+] Building 192.2s (10/10) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.3s
=> => transferring dockerfile: 161B 0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 4.4s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 28 0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:f64cd12c85ee0368f 183.1s
=> => resolve docker.io/library/node:20-alpine@sha256:f64cd12c85ee0368f 0.0s
=> => sha256:f64cd12c85ee0368f6a75362a0b05d5d5c50a0463 7.67kB / 7.67kB 0.0s
=> => sha256:ffdf98210b07b596eb71fa22ba2e4320858d4cd1384 1.72kB / 1.72kB 0.0s
=> => sha256:11ced39e663e7c5d5cb9cc77a461a0d2ad25537b9 6.52kB / 6.52kB 0.0s
=> => sha256:16a0ac1617061a677b045b7ff08545213ec31c0ff087 3.06MB / 3.06MB 8.4s
=> => sha256:4f6ee04c154301db6f4a496efa397b3db96603b 43.23MB / 43.23MB 100.4s
=> => sha256:b2cbbfe903b0821005780971ddc5092edcc4ce74c5a 1.26MB / 1.26MB 2.2s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8b 445B / 445B 2.5s
=> => extracting sha256:16a0ac1617061a677b045b7ff08545213ec31c0ff08763195 0.2s
=> => extracting sha256:4f6ee04c154301db6f4a496efa397b3db96603b1c009c797 2.2s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5092edcc4ce74c5a48d82 0.1s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 807B 0.0s
=> [2/5] WORKDIR /app 0.2s
=> [3/5] COPY package*.json ./ 0.0s
=> [4/5] RUN npm install 3.7s
=> [5/5] COPY . . 0.0s
=> => exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:9a560556d7a179445be7caced83b1e0d4884f7a29cf461 0.0s
=> => naming to docker.io/library/my-node-app:v1 0.0s
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app$ docker images
```

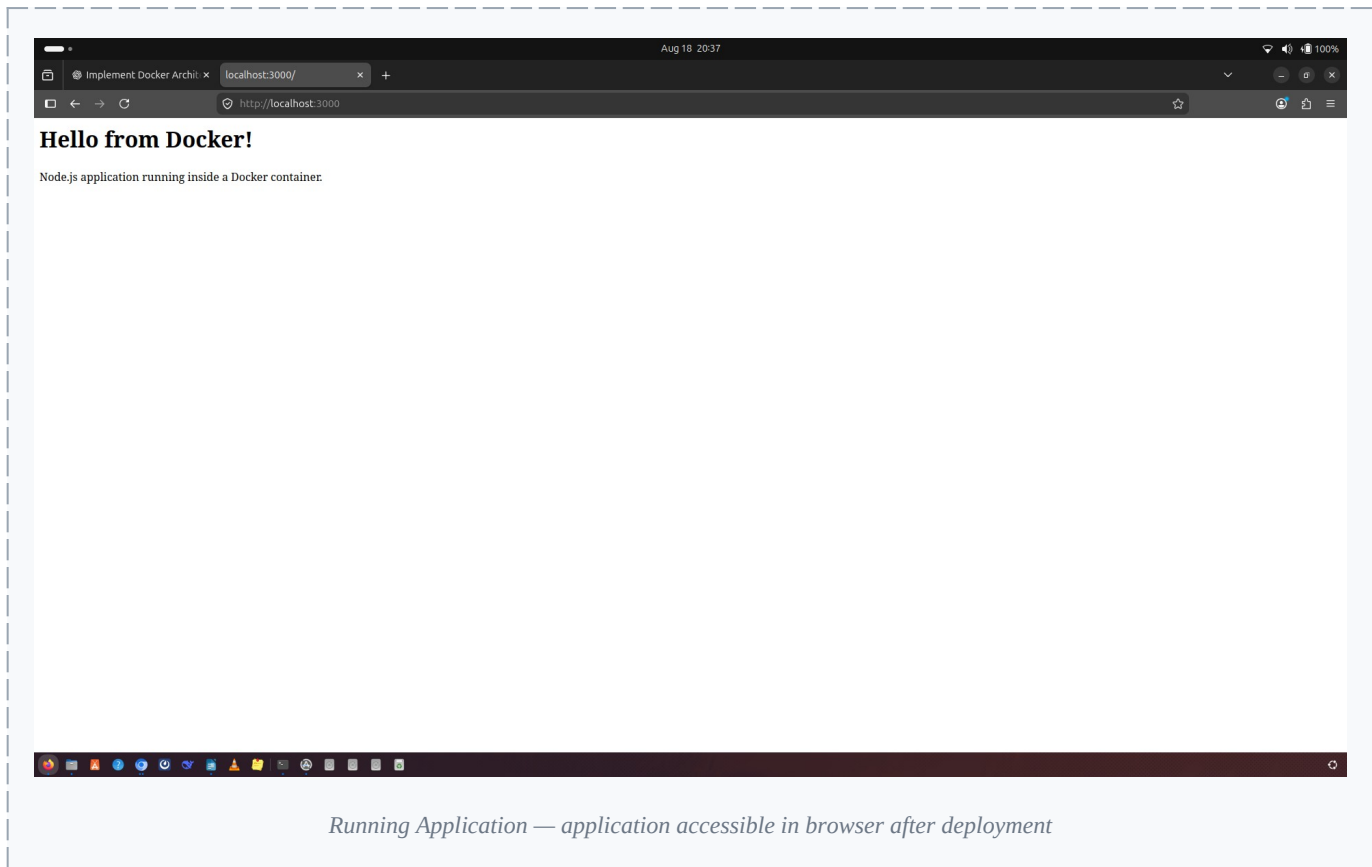
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
las-project-react-front-app:latest	dc99cd366f7b	4.42GB	0B	
laine11/phpldapadmin:1.2.6.6	5303ea077f1d	229MB	0B	
mongo-express:latest	870141b735e7	182MB	0B	
mongo:latest	f0286cc18189	942MB	0B	
my-node-app:v1	9a560556d7a1	136MB	0B	
nginx:alpine	b76de378d572	62.1MB	0B	

```
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app$ docker run -d --name node-app -p 3000:3000 my-node-app:v1
c153dce49a94d7b226b65048c557aae2a673f3bec4dce8427acd3709e10da384
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c153dce49a94	my-node-app:v1	"docker-entrypoint.s..."	15 seconds ago	Up 14 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	node-app
b67685bebcc6	nginx	"/docker-entrypoint..."	37 minutes ago	Up 37 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	nginx-container-2

```
eng-kiipnetich@eng-kiipnetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker/node-app$
```

Docker Hub Repository — pushed image visible in Docker Hub



Running Application — application accessible in browser after deployment

10. Multi-Container Application Summary

For the hands-on activity, a multi-container application was deployed consisting of a frontend, backend, and database, connected via a custom Docker network and using a Docker volume for persistent database storage. All services were orchestrated using a single docker-compose.yml file and verified to communicate successfully.

11. Docker Best Practices Applied

- Used lightweight, minimal base images (e.g. alpine variants) to reduce image size.
- Added a .dockerignore file to exclude unnecessary files (e.g. node_modules, .git) from the build context.
- Used named volumes for persistent data instead of writing to the container's writable layer.
- Used a custom bridge network so containers could resolve each other by service name.
- Kept environment-specific configuration (credentials, hosts) in environment variables rather than hardcoding them.
- Verified AI-generated Dockerfile and Compose configurations by testing them directly in the Docker environment before relying on them.


```
Aug 18 20:42
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker$ docker volume create app_data
app_data
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker$ docker run -it --name volume-test -v app_data:/data alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
/ # echo "Docker volume persistence test" > /data/test.txt
/ # cat /data/test.txt
Docker volume persistence test
/ # ls -l /data
total 4
-rw-r--r-- 1 root root 31 Aug 18 17:41 test.txt
/ #
```

```
Aug 18 20:39
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB: ~/DevOps/week4-docker
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker/node-app$ docker run -d --name node-app -p 3000:3000 my-node-app:v1
c153dce49a94d7b226b65048c557a2a673f3bec4dce8427acd3709e10da384
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker/node-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
c153dce49a94   my-node-app:v1 "docker-entrypoint.s..." 15 seconds ago Up 14 seconds 0.0.0.0:3000->3000/tcp             node-app
b67685bebccc   nginx          "/docker-entrypoint..." 37 minutes ago Up 37 minutes 0.0.0.0:8081->80/tcp               nginx-container-2
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker/node-app$ cd ..
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker$ docker volume create app_data
app_data
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker$ docker volume ls
DRIVER      VOLUME NAME
local      app_data
eng-kipngetich@eng-kipngetich-Lenovo-YOGA-520-14IKB:~/DevOps/week4-docker$
```

12. Implementation Summary

The project involved containerizing a Node.js application using Docker, with the application configured to listen on port 3000 and accept connections through 0.0.0.0. The Dockerfile was structured using the Node.js 20 Alpine base image, installed the required dependencies, copied the application files, exposed port 3000, and started the application with `server.js`. During implementation, a port conflict was encountered because another container was already using port

3000; this was resolved by stopping the conflicting container before starting the Node.js container. The container was successfully built and started, and the application was verified through the browser at <http://localhost:3000>, confirming successful container execution and network communication.

	Status
Dockerfile	Done
Docker Compose File	Done
Screenshot — Docker Installation	Done
Screenshot — Running Containers	Done
Screenshot — Docker Images	Done
Screenshot — Docker Hub Repository	https://app.docker.com/accounts/engkipngetich

Prepared as part of the Davine Technologies DevOps Internship — Week 4: Docker & Docker Compose.